UPDATED GIT & DEVOPS PRACTICAL GUIDE

Experiments 1–8: Windows | Experiments 9–12: Ubuntu

Platform split: Perform Experiments 1–8 on Windows. Perform Experiments 9–12 on Ubuntu. For file contents, create the named file in VS Code and paste the code block. For shell commands, paste into the terminal specified.

Platform and terminal rules

Windows Git tasks: Git Bash. Windows Python: VS Code Terminal / PowerShell. Experiment 5: GitHub Desktop. Experiment 8: Windows Jenkins. Experiments 9–12: Ubuntu Terminal, VS Code on Ubuntu, Docker Engine and Minikube.

Use your own GitHub username and repository URLs where shown. Keep passwords, tokens and secrets out of source files.

Experiments 1–8 – Windows

Use the previously supplied PDF **Git_DevOps_Experiments_2_to_12_Guide.pdf** for the step-by-step procedures for Experiments 2–8, and your Experiment 1 PDF for Experiment 1. Execute all those steps on Windows. Experiment 8 uses Windows Jenkins commands and Windows batch steps (**bat**) in its Jenkinsfile.

Do not use the Ubuntu commands below until Experiment 9.

EXPERIMENT 9 – Docker on Ubuntu

Terminal: Ubuntu Terminal. **Files:** create in VS Code on Ubuntu.

Step 1 – Install Docker Engine and Compose plugin

Paste in: Ubuntu Terminal

```
sudo apt update
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu $(
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
sudo systemctl enable --now docker
sudo docker --version
sudo docker compose version
```

Step 2 – Allow Docker commands without sudo (optional)

Paste in: Ubuntu Terminal

```
sudo usermod -aG docker $USER
newgrp docker
docker ps
```

Step 3 – Create project

Paste in: Ubuntu Terminal

```
mkdir -p ~/DockerAIApp
cd ~/DockerAIApp
code .
```

Step 4 – Create app.py

Paste in: VS Code – create ~/DockerAIApp/app.py

```
from flask import Flask, jsonify

app = Flask(__name__)

@app.route("/")
def home():
    return "AI demo application is running"

@app.route("/predict")
def predict():
    return jsonify({"prediction": "demo result", "status": "success"})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Step 5 – Create requirements.txt

Paste in: VS Code – create ~/DockerAIApp/requirements.txt

```
flask
```

Step 6 – Create Dockerfile

Paste in: VS Code – create ~/DockerAIApp/Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

Step 7 – Build image and run container

Paste in: Ubuntu Terminal – ~/DockerAIApp

```
docker build -t ai-demo:1.0 .
docker run -d --name ai-demo-container -p 5000:5000 ai-demo:1.0
docker ps
```

Step 8 – Test and stop

Paste in: Ubuntu Terminal / Firefox browser

```
curl http://localhost:5000/
curl http://localhost:5000/predict
docker stop ai-demo-container
docker rm ai-demo-container
```

Expected check: first curl prints the app message; second returns JSON. docker ps shows the running container before stopping it.

EXPERIMENT 10 – Docker Compose on Ubuntu

Terminal: Ubuntu Terminal. **Files:** create in VS Code.

Step 1 – Create project folders

Paste in: Ubuntu Terminal

```
mkdir -p ~/ComposeAIApp/api ~/ComposeAIApp/frontend
cd ~/ComposeAIApp
code .
```

Step 2 – Create API file

Paste in: VS Code – api/app.py

```
from flask import Flask, jsonify

app = Flask(__name__)

@app.route("/health")
def health():
    return jsonify({"api": "ok"})

@app.route("/predict")
def predict():
    return jsonify({"prediction": "demo class", "model": "sample"})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Step 3 – Create API requirements

Paste in: VS Code – api/requirements.txt

```
flask
```

Step 4 – Create API Dockerfile

Paste in: VS Code – api/Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

Step 5 – Create frontend

Paste in: VS Code – frontend/index.html

```
<!doctype html>
<html>
  <body>
    <h1>AI Demo Frontend</h1>
    <a href="http://localhost:5000/predict">Open demo prediction API</a>
  </body>
</html>
```

Step 6 – Create docker-compose.yml

Paste in: VS Code – ~/ComposeAIApp/docker-compose.yml

```
services:
  api:
    build: ./api
    ports:
      - "5000:5000"
    depends_on:
      - db

  frontend:
    image: nginx:alpine
    ports:
      - "8080:80"
    volumes:
      - ./frontend:/usr/share/nginx/html:ro
    depends_on:
      - api

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: aiuser
      POSTGRES_PASSWORD: demo_password
      POSTGRES_DB: aidb
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

Step 7 – Start and inspect containers

Paste in: Ubuntu Terminal – ~/ComposeAIApp

```
docker compose up --build -d
docker compose ps
docker compose logs
```

Step 8 – Test and stop

Paste in: Ubuntu Terminal / Firefox

```
curl http://localhost:5000/health
curl http://localhost:5000/predict
curl http://localhost:8080
docker compose down
```

Use 'docker compose down -v' only if you also want to delete the PostgreSQL volume and its data.

Expected check: frontend is at http://localhost:8080, API at http://localhost:5000, and PostgreSQL runs as the db service.

EXPERIMENT 11 – Minikube Kubernetes on Ubuntu

Terminal: Ubuntu Terminal. This deploys the Docker image from Experiment 9.

Step 1 – Install kubectl

Paste in: Ubuntu Terminal

```
sudo apt update
sudo apt install -y curl
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
kubectl version --client
```

Step 2 – Install Minikube

Paste in: Ubuntu Terminal

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
minikube version
```

Step 3 – Start cluster

Paste in: Ubuntu Terminal

```
minikube start --driver=docker
kubectl get nodes
```

Step 4 – Build and load image

Paste in: Ubuntu Terminal – ~/DockerAIApp

```
cd ~/DockerAIApp
docker build -t ai-demo:1.0 .
minikube image load ai-demo:1.0
```

Step 5 – Create deployment.yaml

Paste in: VS Code – ~/DockerAIApp/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ai-demo
spec:
  replicas: 2
  selector:
    matchLabels:
      app: ai-demo
  template:
    metadata:
      labels:
        app: ai-demo
    spec:
      containers:
      - name: ai-demo
        image: ai-demo:1.0
        imagePullPolicy: IfNotPresent
        ports:
        - containerPort: 5000
```

Step 6 – Create service.yaml

Paste in: VS Code – ~/DockerAIApp/service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: ai-demo-service
spec:
  type: NodePort
  selector:
    app: ai-demo
  ports:
  - port: 5000
    targetPort: 5000
    nodePort: 30080
```

Step 7 – Deploy and verify

Paste in: Ubuntu Terminal – ~/DockerAIApp

```
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
kubectl get deployments
kubectl get pods
kubectl get services
```

Step 8 – Open service and scale

Paste in: Ubuntu Terminal

```
minikube service ai-demo-service --url
kubectl scale deployment ai-demo --replicas=4
kubectl get pods
```

Step 9 – Clean up

Paste in: Ubuntu Terminal

```
kubectl delete -f service.yaml
kubectl delete -f deployment.yaml
minikube stop
```

Expected check: deployment initially creates two pods; scaling changes desired replicas to four. The Minikube service URL opens the Flask demo.

EXPERIMENT 12 – Jenkins + Docker + Kubernetes CI/CD on Ubuntu

Terminal: Ubuntu Terminal. **Files:** VS Code. Jenkins must run on an agent that can access Docker, kubectl and Minikube.

Step 1 – Install Java and Jenkins

Paste in: Ubuntu Terminal

```
sudo apt update
sudo apt install -y fontconfig openjdk-21-jre
java -version
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian-stable binary/" | sudo tee /etc/apt/so
sudo apt update
sudo apt install -y jenkins
sudo systemctl enable --now jenkins
sudo systemctl status jenkins
```

Use the Java version supported by the Jenkins LTS version at installation time. If the repository installation instructions specify a newer supported JRE, follow those official instructions.

Step 2 – Get initial Jenkins password

Paste in: Ubuntu Terminal

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Step 3 – Open Jenkins and install plugins

Paste in: Firefox browser

```
http://localhost:8080
```

Paste the password from Step 2. Select Install suggested plugins. Create the admin user. In Manage Jenkins → Plugins, install Git, GitHub and Pipeline plugins if not already installed.

Step 4 – Give Jenkins access to Docker and Kubernetes tools

Paste in: Ubuntu Terminal

```
sudo usermod -aG docker jenkins
sudo systemctl restart jenkins
sudo -u jenkins docker version
```

Jenkins must also have kubectl and Minikube configured and access to the same Minikube cluster. A simple classroom setup can run a Jenkins agent under your Ubuntu user with the needed permissions. Do not assume the jenkins service user automatically shares your shell PATH, kubeconfig or Minikube profile.

Step 5 – Verify tools as the Jenkins execution user

Paste in: Ubuntu Terminal

```
which docker
which kubectl
which minikube
kubectl get nodes
docker ps
```

Step 6 – Add Jenkinsfile

Paste in: VS Code – repository root

```
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Build Docker Image') {
            steps {
                sh 'docker build -t ai-demo:latest .'
            }
        }

        stage('Load Image into Minikube') {
            steps {
                sh 'minikube image load ai-demo:latest'
            }
        }

        stage('Deploy to Kubernetes') {
            steps {
                sh 'kubectl apply -f deployment.yaml'
                sh 'kubectl apply -f service.yaml'
            }
        }

        stage('Verify Deployment') {
            steps {
                sh 'kubectl rollout status deployment/ai-demo'
                sh 'kubectl get pods'
            }
        }
    }
}
```

Step 7 – Set deployment image tag

Paste in: VS Code – deployment.yaml

```
containers:
- name: ai-demo
  image: ai-demo:latest
  imagePullPolicy: IfNotPresent
  ports:
  - containerPort: 5000
```

Keep the rest of deployment.yaml structure from Experiment 11. The image tag in the Jenkinsfile and deployment.yaml must match.

Step 8 – Commit and push

Paste in: Ubuntu Terminal – repository folder

```
git add Jenkinsfile Dockerfile app.py requirements.txt deployment.yaml service.yaml
git commit -m "Add Jenkins Docker Kubernetes pipeline"
git push
```

Step 9 – Create Jenkins Pipeline from SCM

Paste in: Firefox browser – Jenkins

```
Dashboard → New Item → enter AI-CI-CD → Pipeline → OK
Pipeline → Definition: Pipeline script from SCM
SCM: Git
Repository URL: https://github.com/YOUR_USERNAME/YOUR_REPOSITORY.git
Branch Specifier: */main
Script Path: Jenkinsfile
Save → Build Now
```

Step 10 – Check build and deployment

Paste in: Jenkins browser and Ubuntu Terminal

```
# Jenkins: open build number → Console Output

# Ubuntu Terminal:
kubectl get deployments
kubectl get pods
kubectl get services
minikube service ai-demo-service --url
```

Step 11 – Configure automatic trigger

Paste in: Jenkins browser

```
Job → Configure → Build Triggers → Poll SCM
Schedule:
H/5 * * * *
Save
```

Push a new commit to GitHub and wait for the polling interval. A GitHub webhook requires a Jenkins URL reachable by GitHub; localhost alone is not publicly reachable.

Step 12 – Clean up after demonstration

Paste in: Ubuntu Terminal

```
kubectl delete -f service.yaml
kubectl delete -f deployment.yaml
minikube stop
```

Expected check: Jenkins checks out the GitHub repository, builds the Docker image, loads it into Minikube, applies Kubernetes YAML and verifies the deployment rollout.

QUICK CHECKS – UBUNTU

OS version

Paste in: Ubuntu Terminal

```
lsb_release -a
```

Docker

Paste in: Ubuntu Terminal

```
docker --version
docker compose version
docker ps
```

Jenkins

Paste in: Ubuntu Terminal

```
sudo systemctl status jenkins
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Minikube

Paste in: Ubuntu Terminal

```
minikube status
kubectl get nodes
```

Kubernetes

Paste in: Ubuntu Terminal

```
kubectl get deployments
kubectl get pods
kubectl get services
```

Stop Minikube

Paste in: Ubuntu Terminal

```
minikube stop
```

Stop Docker Compose project

Paste in: Ubuntu Terminal

```
docker compose down
```